Decoupling What from Where: Action-Type Supervision for GUI Grounding in a Small Vision-Language Model

Anonymous Author(s)
Affiliation
Address
`email`

Abstract

A GUI agent has to decide which action to take (click, scroll, type) and where on the screen to take it. Native agent models emit both decisions in one token stream. We ask whether separating them improves grounding, using Qwen2-VL-2B with LoRA on Android in the Wild and Multimodal-Mind2Web. Stage 1 classifies the action type from frozen features; Stage 2 predicts a coordinate and can be conditioned on the type through one of four mechanisms, compared with a flat baseline under matched data, compute, and decoding. With five seeds and paired bootstrap tests over 1,250 examples, an auxiliary action-type loss and an additive learned embedding each raise hit@0.10 by 5 to 6 points over the flat baseline when the action type predicts the target location, and change nothing on a control mix where it does not. A prepended learned action token, the mechanism we originally hypothesized, is no better than the baseline. Intervention experiments explain the difference: the additive embedding can be zeroed at test time without loss, so its benefit is a training effect, whereas the prepended token is used at inference and still does not help. Across training sizes from 300 to 5,000 examples, only the additive mechanism helps consistently. We also document a positional-encoding pitfall: injecting the conditioning through `inputs_embeds` bypasses Qwen2-VL's M-RoPE and produced an apparent ten-point deficit that vanished once the `input_ids` path was preserved.

1 Introduction

A GUI agent maps a screenshot and an instruction to a low-level action: an action type such as *click*, *scroll*, or *type*, and, for spatial actions, a target location. Current native agent models (UI-TARS [Qin et al., 2025], OS-Atlas [Wu et al., 2025c], SeeClick [Cheng et al., 2024], ShowUI [Lin et al., 2025]) produce both in a single autoregressive stream. The two decisions are different kinds of prediction, though. The action type is a small classification problem that depends on the goal and the screen state; the location is a localization problem, in which the model has to find the evidence on the screen that supports the action and commit to a point. When a joint decoder gets the type wrong, it tends to ground consistently within the wrong choice, scrolling past a visible button instead of clicking it. The action is locally coherent and globally wrong, and on multi-step tasks the error compounds because the screen state changes.

This paper asks a narrow question about that failure mode: does telling the grounding model which action it is grounding improve where it grounds? We build a two-stage pipeline around Qwen2-VL-2B [Wang et al., 2024]. Stage 1 predicts the action type from mean-pooled frozen features with a small MLP. Stage 2 fine-tunes the backbone with LoRA [Hu et al., 2022] to emit a coordinate string

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

and receives the action type through one of four conditioning mechanisms: an auxiliary classification loss, a hard-routed action word in the output, an additive learned embedding, and a prepended learned action token. Each is compared with a flat baseline that sees the same data and uses the same optimizer, adapters, and decoding. We started from the hypothesis that the prepended token would be the right mechanism, because it gives the model a dedicated, attendable position for the action type.

The evidence supports the factorization and refutes our hypothesis about the mechanism. On an Android in the Wild mix where the action type predicts the target location, the auxiliary loss and the additive embedding improve hit@0.10 over the flat baseline by $+0.064$ and $+0.054$ (five seeds, paired bootstrap over 1,250 examples, $p < 0.001$), and the two are indistinguishable from each other. On a control mix of taps and swipes, where the type carries no spatial information, neither differs from the baseline. The prepended token is within noise of the baseline and worse than the auxiliary loss. Across training sizes from 300 to 5,000 examples, the additive embedding is the only mechanism that helps at nearly every size; the auxiliary loss helps at some sizes and hurts at one. Intervention experiments on the trained models explain the ranking: the additive embedding can be zeroed at test time with no loss in accuracy, so its benefit comes from training, whereas the prepended token is used at inference and still does not help. The same study surfaced a silent failure. An earlier implementation injected the conditioning through `inputs_embeds`, which bypasses Qwen2-VL's multimodal rotary position computation; that version scored ten points below the baseline, and keeping the `input_ids` path removed the deficit entirely.

Our contributions are: (1) a matched-compute comparison of four conditioning mechanisms against a flat baseline on two GUI datasets, with five-seed paired statistics and a control mix where conditioning should not help; (2) a scaling study from 300 to 5,000 examples with three seeds per size; (3) interventions and attention measurements that separate training-time from inference-time use of the signal; and (4) a positional-encoding pitfall that produced a large false negative.

2 Related Work

GUI grounding and agent models. Mind2Web [Deng et al., 2023] and Android in the Wild [Rawles et al., 2023] supply the web and mobile episodes we train on; AndroidControl [Li et al., 2024] and AndroidWorld [Rawles et al., 2025] extend mobile evaluation. SeeClick [Cheng et al., 2024] argued that grounding is the bottleneck for visual GUI agents and introduced ScreenSpot; ScreenSpot-Pro [Li et al., 2025] raises the resolution. UGround [Gou et al., 2025] and OS-Atlas [Wu et al., 2025c] scale grounding pretraining, and Aguvis [Xu et al., 2025], UI-TARS [Qin et al., 2025], CogAgent [Hong et al., 2024], ScreenAI [Baechler et al., 2024], and Ferret-UI [You et al., 2024b] train agents that emit action and location together, the design our flat baseline follows. GUI-Actor [Wu et al., 2025a] replaces coordinate text with an attention-based action head, UI-R1 [Lu et al., 2026] rewards the action type and the argument separately, RegionFocus [Luo et al., 2025] zooms at test time, and OmniParser [Lu et al., 2024] and Set-of-Mark prompting [Yang et al., 2023] externalize grounding into a parser. SeeAct [Zheng et al., 2024], Ponder & Press [Wang et al., 2025], and Aria-UI [Yang et al., 2025] separate text planning from pixel grounding; our boundary is between the action type and the location. GUI-Libra [Yang et al., 2026] adds action-aware supervision and GUI-AIMA [Zhou et al., 2025] an anchor token, close in spirit to our auxiliary-loss and prepended-token variants, without the mechanism-level interventions we run.

Conditioning and factorized decision-making. SayCan [Ahn et al., 2022] separates what a language model wants to do from what the environment affords; ReAct [Yao et al., 2023] exposes intermediate decisions as text; RT-2 [Brohan et al., 2023] is the reference for emitting actions as tokens. On the same mobile benchmark we use, AutoUI [Zhang and Zhang, 2024], AppAgent [Zhang et al., 2025], DigiRL [Bai et al., 2024], WebRL [Qi et al., 2025], and VSC-RL [Wu et al., 2025b] improve agents by imitation or reinforcement learning without changing how the action type enters the grounding computation. Our auxiliary-loss variant is a multi-task objective in the sense of Caruana [1997].

Grounded and faithful VLMs. Kosmos-2 [Peng et al., 2024], Shikra [Chen et al., 2023], and Ferret [You et al., 2024a] train VLMs to refer to image regions, and POPE [Li et al., 2023] and HallusionBench [Guan et al., 2024] measure whether a model's output is supported by the image.

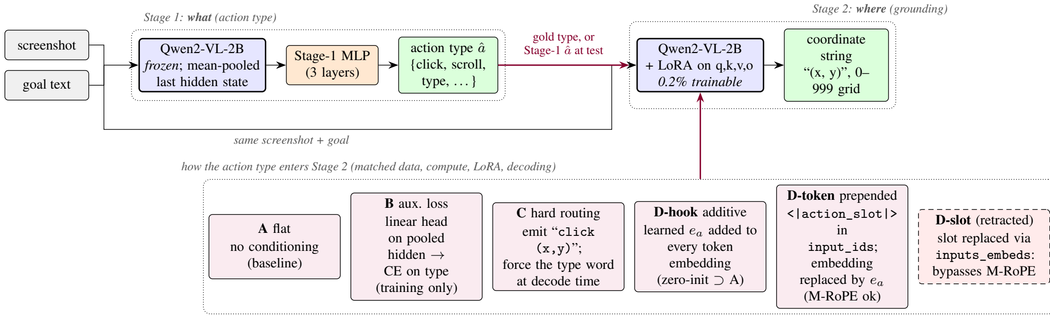

Figure 1: Two-stage pipeline. Stage 1 classifies the action type from mean-pooled frozen features. Stage 2 grounds the action to a coordinate string, and the action type enters it through one of the mechanisms in the bottom row. All mechanisms share data, compute, adapters, and decoding. D-slot is the retracted variant whose `inputs_embeds` injection bypassed M-RoPE.

GUI grounding is a deployment-shaped instance of the same question: the coordinate is only correct if the model found the evidence for it on the screen. Our attention measurements ask directly whether the conditioning signal moves the model's evidence toward the target.

Backbone and adaptation. Qwen2-VL [Wang et al., 2024] assigns three-dimensional rotary positions (M-RoPE, an extension of RoPE [Su et al., 2024]) to visual tokens, and the assignment is keyed off `input_ids`; Section 6.1 shows what happens when conditioning is injected around that path. All fine-tuning uses LoRA [Hu et al., 2022].

3 Method

3.1 Action taxonomy and grounding target

Both datasets are mapped onto a canonical eight-class action space (*click*, *double-click*, *type*, *scroll*, *drag*, *hotkey*, *wait*, *finished*). Mind2Web's CLICK, SELECT, and HOVER map to *click* and TYPE to *type*. AITW stores most actions as dual-point gestures; we split them into taps (*click*) and swipes (*scroll*) by the normalized distance between touch and lift, with threshold 0.04. The grounding target is the touch point normalized to $[0, 1]^2$, serialized as the string `(x, y)` with integers on a 0–999 grid, so every target has the same token length.

3.2 Stage 1: which action

Stage 1 runs the frozen Qwen2-VL-2B-Instruct backbone over the screenshot and instruction, mean-pools the final hidden states over unmasked tokens into a 1536-dimensional vector, and trains a three-layer MLP ($1536 \to 1024 \to 256 \to 8$, dropout 0.1) with class-weighted cross-entropy, AdamW (learning rate 10^{-4}, weight decay 0.01), batch size 128, for eight epochs, keeping the checkpoint with the best validation macro-F1. We compare features from text only, text plus screenshot, and a sanity variant with zeroed features.

3.3 Stage 2: where

Stage 2 fine-tunes the same backbone with LoRA adapters (rank 16, $\alpha = 32$, dropout 0.05) on the query, key, value, and output projections, 4.4M trainable parameters out of 2.2B (0.2%). The prompt is the screenshot followed by `Goal: {goal}` and a fixed instruction to predict the action coordinate; the answer is the coordinate string. Let s be the screenshot, g the goal, a the action type, and $y_{1:T}$ the coordinate tokens. Every variant minimizes

$$\mathcal{L}_{\text{coord}} = -\sum_{t=1}^{T} \log p_\theta\big(y_t \mid y_{<t},\, s,\, g,\, c\big), \tag{1}$$

with the loss masked to the answer tokens and c the variant-specific conditioning.

A, flat. $c = \varnothing$. This is the joint-decoding design of native agent models, reduced to the grounding sub-problem.

B, auxiliary loss. A linear head h_ϕ on the mean-pooled last hidden state $\bar{z}$ predicts the action type during training, $\mathcal{L}_{\mathrm{B}} = \mathcal{L}_{\mathrm{coord}} + \lambda \, \mathrm{CE}(h_\phi(\bar{z}), a)$ with $\lambda = 1$. Nothing about the action type is provided at inference, so any gain is a training effect.

C, hard routing. The model is trained to emit the action word before the coordinate, `click (x, y)`. At test time the word for the gold (or Stage-1) type is forced as a decoding prefix and the model completes the coordinate.

D-hook, additive embedding. A learned table $E \in \mathbb{R}^{8\times d}$ holds one vector per class. A forward hook on the embedding layer adds $E[a]$ to every token embedding. E is zero-initialized, so D-hook starts as an exact copy of A and can only depart from it by learning to use the signal.

D-token, prepended action token. A new token `<|action_slot|>` is added to the vocabulary and placed at the start of the text prompt, so it occupies a real position in `input_ids` and M-RoPE treats it like any other token. A forward hook replaces the embedding at that position with $E[a]$. This is the architecture we originally hypothesized: a full-norm, attendable representation of the action type.

For C, D-hook, and D-token the gold type is used during training and at evaluation unless stated otherwise; Section 5.4 closes the loop with Stage-1 predictions. All variants share the data order, the optimizer (AdamW, learning rate 2×10^{-5}, weight decay 0.01, batch size 1, gradient clipping at 1.0, two epochs), and greedy decoding with 16 new tokens.

4 Experimental setup

Data. Stage 1 uses 7,775 Multimodal-Mind2Web training steps evaluated on the 1,338-step `test_task` split, and 5,000 AITW steps evaluated on 1,000 held-out steps. Stage 2 uses two AITW mixes. `all_with_coords` contains taps, swipes, and type events; the validation slice of 250 steps has 169 clicks, 46 scrolls, and 35 type events, so the action type is informative about location. `taps_and_swipes` contains only taps and swipes (1,000 training and 200 validation steps); both land anywhere on the screen, so the type carries no spatial information; this mix is the control. A second control uses Multimodal-Mind2Web (1,000 training and 250 evaluation steps), where the target is the center of the gold element and we also report whether the predicted point falls inside the element's box. The AITW stream is deterministic, so for a given mix and training size every seed and every variant is evaluated on the same examples in the same order. The validation slice starts where the training slice ends, which means absolute numbers are not comparable across training sizes; only within-size differences are.

Metrics and statistics. hit@r is the fraction of predictions within normalized Euclidean distance r of the target, for $r \in \{0.05, 0.10, 0.25\}$; hit@0.10 is the headline. We also report the mean normalized distance. Unparseable outputs count as misses at distance $\sqrt{2}$. The headline mix uses five seeds (42 to 46), all other cells three (42 to 44); we report mean and sample standard deviation over seeds. Because every run logs a per-example distance on a shared evaluation set, we test differences with a paired bootstrap over pooled (seed, example) units, matching seeds by id: 10,000 resamples, 95% percentile intervals, and a two-sided bootstrap p-value [Efron and Tibshirani, 1993]. A paired permutation test agrees in every case we report. Stars denote $p < 0.05$, 0.01, and 0.001.

5 Results

5.1 Stage 1: the screenshot decides the action type on AITW

Stage 1 (Table 5 in the appendix, three seeds) sets up the rest of the paper. On Mind2Web the instruction usually contains the action word: a bag-of-words classifier reaches 0.622 macro-F1, the frozen VLM features 0.605, and the screenshot is worth $+0.009$. On AITW the instruction is a goal ("open a new private window"), text-only features sit at the majority-class floor (0.141), and the screenshot is worth $+0.414$ macro-F1 (0.555). Predicting the action type is a vision problem on AITW, which is why the Stage-2 study is run there and why Mind2Web is used as a control.

Table 1: Stage 2 grounding on AITW `all_with_coords` (n_{train}=1200, n_{val}=250, 5 seeds). Mean ± std over seeds; Δ with 95% paired-bootstrap CI over pooled (seed, example) units; $^{*}p < .05$, $^{**}p < .01$, $^{***}p < .001$.

Variant	hit@0.05	hit@0.10	hit@0.25	mean L2 ↓	Δhit@0.10 vs. A [cluster CI]	seed-level p
A: flat	0.130 ± 0.039	0.229 ± 0.043	0.499 ± 0.029	0.406 ± 0.028		
B: aux. loss	0.178 ± 0.025	0.293 ± 0.021	0.582 ± 0.026	0.365 ± 0.004	+0.064 [+0.036, +0.093]***	0.004
C: hard routing	0.170 ± 0.022	0.264 ± 0.045	0.538 ± 0.059	0.421 ± 0.016	+0.035 [+0.001, +0.069]*	0.189
D-hook: additive	0.170 ± 0.042	0.282 ± 0.041	0.563 ± 0.016	0.370 ± 0.010	+0.054 [+0.022, +0.086]***	0.028
D-token: prepended	0.144 ± 0.056	0.238 ± 0.049	0.504 ± 0.055	0.386 ± 0.021	+0.009 [-0.015, +0.034]	0.465

Table 2: Control: AITW `taps_and_swipes` (n_{train}=1000, n_{val}=200, 3 seeds), where action type is not spatially informative.

Variant	hit@0.05	hit@0.10	hit@0.25	mean L2 ↓	Δhit@0.10 vs. A [cluster CI]	seed-level p
A: flat	0.243 ± 0.020	0.382 ± 0.015	0.720 ± 0.017	0.185 ± 0.003		
B: aux. loss	0.225 ± 0.015	0.368 ± 0.012	0.730 ± 0.039	0.173 ± 0.011	-0.013 [-0.038, +0.012]	0.456
C: hard routing	0.217 ± 0.016	0.325 ± 0.040	0.635 ± 0.064	0.271 ± 0.068	-0.057 [-0.108, -0.007]*	0.215
D-hook: additive	0.235 ± 0.022	0.363 ± 0.021	0.725 ± 0.043	0.182 ± 0.009	-0.018 [-0.052, +0.015]	0.334

5.2 Conditioning helps where the action type predicts the location

Table 1 is the main result: five variants on `all_with_coords` at 1,200 training examples, five seeds each. The auxiliary loss (B) and the additive embedding (D-hook) both beat the flat baseline on every metric, by $+0.064$ and $+0.054$ hit@0.10 with intervals that exclude zero at $p < 0.001$, and they are statistically tied with each other (D-hook − B: -0.010 hit@0.10, 95% CI $[-0.033, +0.012]$). Hard routing (C) gains less ($+0.035$, $p < 0.01$) and loses on mean distance, partly because forcing the action word makes 2 to 8% of outputs unparseable. The prepended token (D-token), our hypothesized mechanism, is within noise of the baseline on every hit rate ($+0.009$ hit@0.10, CI $[-0.012, +0.028]$) and is worse than B by -0.055 hit@0.10 ($p < 0.001$). Seed variance is large: the baseline ranges from 0.169 to 0.280 across seeds, which is why we report five seeds and paired statistics rather than a best run. The ordering with three seeds was the same; adding two seeds widened the gap between D-token and the two working mechanisms.

The per-class breakdown (Table 6 in the appendix) shows where the gain comes from. Every variant scores exactly zero on *type* because AITW records type events with a pinned coordinate at distance $\sqrt{2}$ from anything a model would predict; the class is a constant across variants, not a failure of any of them. The improvement is concentrated on clicks: B and C gain about ten points there ($0.256 \rightarrow 0.357$ and 0.356), and C pays for its click gain with a thirteen-point loss on scrolls, because forcing the word *scroll* into the output derails the gesture. D-hook is the only variant that improves both clicks (0.321) and scrolls ($0.304 \rightarrow 0.357$), which anticipates its behavior across training sizes below.

5.3 Control: no gain where the type is uninformative

On `taps_and_swipes` (Table 2) the same mechanisms do nothing or harm. B and D-hook are within noise of the baseline on every hit rate (-0.013 and -0.018 hit@0.10, both ns), and C is worse by -0.057 ($p < 0.01$) with a large increase in mean distance. Multimodal-Mind2Web, where clicks dominate and the target is an element center, behaves the same way: with three seeds, D-hook against A is -0.011 hit@0.10, $+0.019$ hit@0.25, and $+0.009$ on the fraction of points inside the gold element ($0.095 \rightarrow 0.104$), none distinguishable from zero. The pattern across the three settings is what the factorization predicts: action-type supervision improves grounding when the type carries information about the target, and is inert when it does not.

5.4 End to end: predicted types instead of gold

The tables above condition on the gold action type. To close the loop we train D-hook, extract frozen features for its training and validation examples, fit the Stage-1 MLP on the three classes present, and evaluate D-hook twice on the same examples: once with gold types and once with Stage-1 predictions (three seeds). Stage 1 reaches 0.843 accuracy and 0.795 macro-F1 in this setting. The oracle pipeline scores 0.292 hit@0.10 and the predicted-type pipeline 0.271, against 0.255 for the

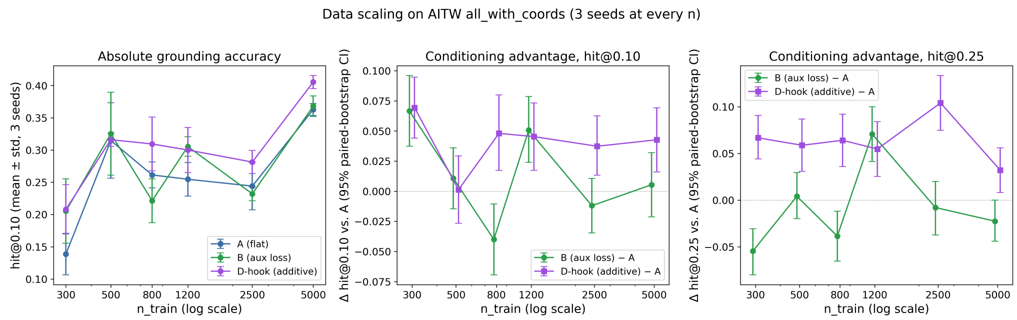

Figure 2: Conditioning advantage over the flat baseline as a function of training-set size on `all_with_coords`, three seeds per cell, 95% paired-bootstrap intervals. Absolute accuracy (left) is not comparable across sizes because the validation slice moves with n; the within-size differences (center, right) are. D-hook is above zero at five of six sizes on hit@0.10 and at all six on hit@0.25; B is above zero at two sizes and below at one. Full numbers in Table 7.

flat baseline trained on the same seeds. The oracle-to-predicted gap is 0.021 ± 0.008, so classifier error removes about a third of the conditioning gain but does not erase it.

5.5 Scaling: only the additive mechanism helps at every size

Figure 2 extends the comparison to $n \in \{300, 500, 800, 1200, 2500, 5000\}$ with three seeds at every size. Two things change relative to the headline. First, the advantage of the auxiliary loss is not a general property: B beats A at 300 ($+0.067$) and 1,200 ($+0.051$), is indistinguishable from A at 500, 2,500, and 5,000, and is worse at 800 (-0.040, $p < 0.01$). Second, D-hook's advantage persists: $+0.069$, $+0.001$, $+0.048$, $+0.045$, $+0.037$, and $+0.043$ hit@0.10 from smallest to largest, significant everywhere except at 500, and positive at every size on hit@0.25. An earlier single-seed version of this curve suggested that the conditioning advantage shrinks with data; with three seeds, that is true of B and not of D-hook, whose gain at 5,000 examples ($+0.043$) is the same size as at 1,200. The two mechanisms tie at the headline size and differ everywhere else, and the difference favors the one that carries the action type into the forward pass.

Figure 4 in the appendix shows what the numbers look like on individual screens at the headline setting. Conditioning rescues some clicks that the baseline sends off-screen, and it also moves some correct clicks onto the wrong list item; on that seed the rescues outnumber the harms by 18 to 11 among 169 clicks, 45 clicks are correct under both models, and 90 are missed by both.

6 Mechanism

6.1 A positional-encoding pitfall that produced a false negative

Our first implementation of the prepended token replaced the slot's embedding by constructing `inputs_embeds` and passing that to the model instead of `input_ids`. On `taps_and_swipes` that variant scored 0.298 ± 0.026 hit@0.10 against 0.390 ± 0.010 for the flat baseline, a nine-point deficit with non-overlapping seeds, and we initially took it as evidence against conditioning. A diagnostic variant with the same slot but a frozen random embedding scored the same (0.290), which pointed at the injection path rather than the learned vector. Qwen2-VL computes its three-dimensional rotary positions from `input_ids`, locating the image tokens in the id sequence to assign temporal, height, and width positions. Given `inputs_embeds` instead, that computation does not see the same sequence, and the visual tokens are positioned as if the slot were not there, which corrupts the spatial layout the task depends on. D-hook and D-token keep `input_ids` intact and modify embeddings through a forward hook; with that change the additive variant scored 0.392 ± 0.043 on the same control, a tie with the baseline, and the headline results followed. Any conditioning scheme that assembles `inputs_embeds` for a model with multimodal positional encoding should verify that the position computation is unaffected, because the failure is silent and, in a two-billion-parameter model, large enough to invert a conclusion.

Table 3: D-token causal-use test: the same trained model evaluated with the gold action embedding, a wrong (cyclically permuted) one, and a zeroed one (5 seeds, paired bootstrap over pooled examples).

Conditioning	hit@0.05	hit@0.10	hit@0.25	mean L2 ↓
gold	0.140 ± 0.059	0.236 ± 0.058	0.502 ± 0.045	0.386 ± 0.020
wrong	0.014 ± 0.004	0.079 ± 0.013	0.200 ± 0.027	0.728 ± 0.081
zero	0.074 ± 0.055	0.167 ± 0.066	0.416 ± 0.103	0.489 ± 0.059
gold − wrong		+0.157 [+0.131, +0.182]***	+0.302 [+0.273, +0.331]***	-0.342 [-0.360, -0.324]***
gold − zero		+0.069 [+0.045, +0.092]***	+0.086 [+0.062, +0.109]***	-0.103 [-0.116, -0.090]***

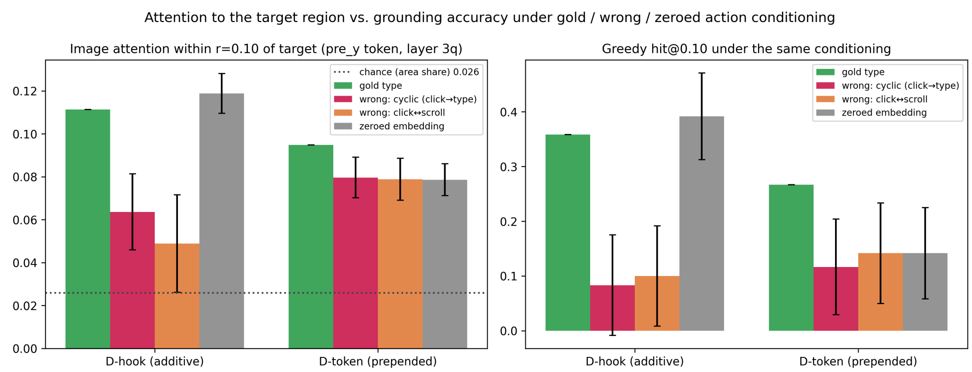

Figure 3: Same trained model, four conditionings, 120 validation screens, one seed per variant. Left: share of image attention within 0.10 of the target from the token that predicts the y coordinate (3/4-depth layer, head-averaged); the dotted line is the share of image tokens within that radius. Right: greedy hit@0.10 under the same conditioning. Error bars are 95% paired-bootstrap intervals of the difference from gold. Zeroing the additive embedding changes nothing; zeroing the prepended token costs accuracy and attention.

6.2 Is the learned action embedding used at inference?

D-token's failure to help could mean the model ignores the slot. Table 3 rules that out. We evaluate each trained D-token model (five seeds, 1,250 pooled examples) three times on the same validation set: with the gold action embedding, with a wrong one obtained by cyclically permuting the classes present (click to type, type to scroll, scroll to click), and with the embedding table zeroed. The wrong embedding drops hit@0.10 from 0.236 to 0.079 and the zeroed one to 0.167; both differences exclude zero at $p < 0.001$. The learned vector has norm 2.2 and the model obeys it. The cyclic permutation sends most examples (clicks) to *type*, whose training targets are the degenerate coordinate, so part of the wrong-embedding drop is the model dutifully emitting that coordinate; the zeroed condition has no such confound and still costs seven points. Section 6.3 adds a wrong map that swaps only clicks and scrolls and finds a similar collapse. D-token is therefore used and not enough: the refutation is of the mechanism's usefulness, not of its activity.

6.3 Where the model looks, and whether the signal is needed at inference

To separate training-time from inference-time effects we repeat the intervention for D-hook, add a second wrong map that swaps clicks and scrolls (and sends the ungroundable type class to click), and measure attention at the same time (Figure 3, Table 4). For attention we teacher-force the gold answer and read the head-averaged attention row of the token whose next prediction is the first digit of the y coordinate, at the layer three quarters of the way through the network; the share of that row's image mass falling within 0.10 of the target is our localization measure, and the share of image tokens within that radius (0.026) is chance. Under gold conditioning, both models put 10 to 11% of that token's image attention on the target region, four times chance, so the coordinate is grounded in the right part of the screen when it is right. The last prompt token, which earlier visualizations used, localizes far less (about 4%) under every conditioning; localization happens once the x coordinate has been emitted.

Table 4: Where the model looks, and whether it matters. Share of the y-predicting token's image attention within 0.10 of the gold target (3/4-depth layer; chance is 0.026) and greedy hit@0.10 under gold, wrong and zeroed action conditioning on the first 120 validation examples, one seed per model. Δ = gold $-$ condition with 95% paired-bootstrap CI.

Model	Conditioning	target attn.	Δ vs. gold	hit@0.10	Δ vs. gold
D-hook (additive)	gold type	0.111		0.358	
	wrong (cyclic)	0.064	+0.048 [+0.030, +0.065]***	0.083	+0.275 [+0.183, +0.367]***
	wrong (click$\leftrightarrow$scroll)	0.049	+0.062 [+0.040, +0.086]***	0.100	+0.258 [+0.167, +0.350]***
	zeroed	0.119	-0.008 [-0.017, +0.001]	0.392	-0.033 [-0.117, +0.042]
D-token (prepended)	gold type	0.095		0.267	
	wrong (cyclic)	0.080	+0.015 [+0.006, +0.025]**	0.117	+0.150 [+0.058, +0.233]**
	wrong (click$\leftrightarrow$scroll)	0.079	+0.016 [+0.006, +0.026]**	0.142	+0.125 [+0.033, +0.217]**
	zeroed	0.079	+0.016 [+0.009, +0.024]***	0.142	+0.125 [+0.042, +0.208]**

The two mechanisms diverge on the interventions. For D-hook, zeroing the embedding leaves attention (0.119 vs. 0.111) and accuracy (0.392 vs. 0.358) unchanged or slightly higher, while either wrong map roughly halves the target attention and drops hit@0.10 to 0.08 to 0.10. The model reads the additive signal, and it has also learned, through the LoRA weights, to do without it; the benefit is a training effect and the signal is redundant at test time unless it lies. For D-token, zeroing costs both attention ($0.095 \rightarrow 0.079$, $p < 0.001$) and accuracy ($0.267 \rightarrow 0.142$, $p < 0.01$), as much as a wrong type does. The prepended token is consulted at inference, and consulting it does not make the model better than a baseline that never saw an action type. Together with Section 5.5, this explains the ranking: the value of action-type supervision here is what it teaches the adapter weights, and mechanisms that deliver it as a training signal (B) or as a redundant additive one (D-hook) capture it, while a mechanism that makes the model depend on the signal at inference (D-token) does not.

7 Limitations

The study is deliberately small. One backbone at 2B parameters, LoRA adapters only, validation slices of 200 to 250 steps, and one L4 per run mean that the absolute grounding numbers are far below the field's leaderboards and that per-seed variance is large; the conclusions rest on paired statistics over pooled examples rather than on any single run. AITW's type events are ungroundable, so the "spatially informative" mix is in effect a click-versus-scroll-versus-type discrimination. The attention analysis in Section 6.3 uses one seed per variant and one layer; the y-predicting token was chosen after observing that the last prompt token does not localize, and all positions are reported in the appendix. Mind2Web is a control rather than a target, and we do not evaluate on ScreenSpot or on long-horizon task success, where the compounding-error argument of the introduction would have to be tested directly. The mechanisms are compared at one learning rate and one adapter configuration; a stronger D-token could exist under a different schedule for the embedding table.

8 Conclusion

Telling a small grounding model which action it is grounding helps when the action type says something about where the target is, and does nothing when it does not. The help comes from training, not from the signal at inference: an auxiliary loss captures it at the headline size, an additive embedding that the model can ignore at test time captures it at every size we tried, and the prepended token we set out to validate is used by the model and improves nothing. For practitioners adapting small VLMs as GUI agents, the lessons are to supervise the action type during adaptation, to prefer conditioning the model does not depend on at inference, and to check position ids whenever `inputs_embeds` is on the path.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Mengyuan Yan, and Andy Zeng. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2022.

Gilles Baechler, Srinivas Sunkara, Maria Wang, Fedir Zubach, Hassan Mansoor, Vincent Etter, Victor Cărbune, Jason Lin, Jindong Chen, and Abhanshu Sharma. ScreenAI: A vision-language model for UI and infographics understanding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.

Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. DigiRL: Training in-the-wild device-control agents with autonomous reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023.

Rich Caruana. Multitask learning. *Machine Learning*, 28(1):41–75, 1997. doi: 10.1023/A:1007379606734.

Keqin Chen, Zhao Zhang, Weili Zeng, Richong Zhang, Feng Zhu, and Rui Zhao. Shikra: Unleashing multimodal LLM's referential dialogue magic. arXiv preprint arXiv:2306.15195, 2023.

Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Yantao Li, Jianbing Zhang, and Zhiyong Wu. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9313–9332, 2024. doi: 10.18653/v1/2024.acl-long.505.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Number 57 in Monographs on Statistics and Applied Probability. Chapman & Hall, New York, 1993.

Boyu Gou, Ruohan Wang, Boyuan Zheng, Yanan Xie, Cheng Chang, Yiheng Shu, Huan Sun, and Yu Su. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *International Conference on Learning Representations (ICLR)*, 2025.

Tianrui Guan, Fuxiao Liu, Xiyang Wu, Ruiqi Xian, Zongxia Li, Xiaoyu Liu, Xijun Wang, Lichang Chen, Furong Huang, Yaser Yacoob, Dinesh Manocha, and Tianyi Zhou. HallusionBench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14375–14385, 2024. doi: 10.1109/CVPR52733.2024.01363.

Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. CogAgent: A visual language model for GUI agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14281–14290, 2024. doi: 10.1109/CVPR52733.2024.01354.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Kaixin Li, Ziyang Meng, Hongzhan Lin, Ziyang Luo, Yuchen Tian, Jing Ma, Zhiyong Huang, and Tat-Seng Chua. ScreenSpot-Pro: GUI grounding for professional high-resolution computer use. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM)*, pages 8778–8786, 2025. doi: 10.1145/3746027.3755688.

Wei Li, William Bishop, Alice Li, Chris Rawles, Folawiyo Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. On the effects of data scale on UI control agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 292–305, 2023. doi: 10.18653/v1/2023.emnlp-main.20.

Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. ShowUI: One vision-language-action model for GUI visual agent. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19498–19508, 2025. doi: 10.1109/CVPR52734.2025.01816.

Yadong Lu, Jianwei Yang, Yelong Shen, and Ahmed Awadallah. OmniParser for pure vision based GUI agent. arXiv preprint arXiv:2408.00203, 2024.

Zhengxi Lu, Yuxiang Chai, Yaxuan Guo, Xi Yin, Liang Liu, Hao Wang, Han Xiao, Shuai Ren, Pengxiang Zhao, Guangyi Liu, Guanjing Xiong, and Hongsheng Li. UI-R1: Enhancing efficient action prediction of GUI agents by reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 17608–17616, 2026. doi: 10.1609/aaai.v40i21.38816.

Tiange Luo, Lajanugen Logeswaran, Justin Johnson, and Honglak Lee. Visual test-time scaling for GUI agent grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

Zhiliang Peng, Wenhui Wang, Li Dong, Yaru Hao, Shaohan Huang, Shuming Ma, and Furu Wei. Kosmos-2: Grounding multimodal large language models to the world. In *International Conference on Learning Representations (ICLR)*, 2024.

Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Wenyi Zhao, Yu Yang, Xinyue Yang, Jiadai Sun, Shuntian Yao, Tianjie Zhang, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2025.

Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. UI-TARS: Pioneering automated GUI interaction with native agents. arXiv preprint arXiv:2501.12326, 2025.

Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillicrap. Android in the wild: A large-scale dataset for Android device control. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2025.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063.

Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. arXiv preprint arXiv:2409.12191, 2024.

Yiqin Wang, Haoji Zhang, Jingqi Tian, and Yansong Tang. Ponder & Press: Advancing visual GUI agent towards general computer control. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 1461–1473, 2025. doi: 10.18653/v1/2025.findings-acl.76.

Qianhui Wu, Kanzhi Cheng, Rui Yang, Chaoyun Zhang, Jianwei Yang, Huiqiang Jiang, Jian Mu, Baolin Peng, Bo Qiao, Reuben Tan, Si Qin, Lars Liden, Qingwei Lin, Huan Zhang, Tong Zhang, Jianbing Zhang, Dongmei Zhang, and Jianfeng Gao. GUI-Actor: Coordinate-free visual grounding for GUI agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.

Qingyuan Wu, Jianheng Liu, Jianye Hao, Jun Wang, and Kun Shao. Advancing autonomous VLM agents via variational subgoal-conditioned reinforcement learning. arXiv preprint arXiv:2502.07949, 2025b. Presented at the NeurIPS 2025 Workshop on Vision Language Models: Challenges of Real World Deployment.

Zhiyong Wu, Zhenyu Wu, Fangzhi Xu, Yian Wang, Qiushi Sun, Chengyou Jia, Kanzhi Cheng, Zichen Ding, Liheng Chen, Paul Pu Liang, and Yu Qiao. OS-ATLAS: A foundation action model for generalist GUI agents. In *International Conference on Learning Representations (ICLR)*, 2025c.

Yiheng Xu, Zekun Wang, Junli Wang, Dunjie Lu, Tianbao Xie, Amrita Saha, Doyen Sahoo, Tao Yu, and Caiming Xiong. Aguvis: Unified pure vision agents for autonomous GUI interaction. In *International Conference on Machine Learning (ICML)*, 2025.

Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. arXiv preprint arXiv:2310.11441, 2023.

Rui Yang, Qianhui Wu, Zhaoyang Wang, Hanyang Chen, Ke Yang, Hao Cheng, Huaxiu Yao, Baolin Peng, Huan Zhang, Jianfeng Gao, and Tong Zhang. GUI-Libra: Training native GUI agents to reason and act with action-aware supervision and partially verifiable RL. arXiv preprint arXiv:2602.22190, 2026.

Yuhao Yang, Yue Wang, Dongxu Li, Ziyang Luo, Bei Chen, Chao Huang, and Junnan Li. Aria-UI: Visual grounding for GUI instructions. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 22418–22433, 2025. doi: 10.18653/v1/2025.findings-acl.1152.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Haoxuan You, Haotian Zhang, Zhe Gan, Xianzhi Du, Bowen Zhang, Zirui Wang, Liangliang Cao, Shih-Fu Chang, and Yinfei Yang. Ferret: Refer and ground anything anywhere at any granularity. In *International Conference on Learning Representations (ICLR)*, 2024a.

Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. Ferret-UI: Grounded mobile UI understanding with multimodal LLMs. In *European Conference on Computer Vision (ECCV)*, pages 240–255, 2024b. doi: 10.1007/978-3-031-73039-9_14.

Chi Zhang, Zhao Yang, Jiaxuan Liu, Yanda Li, Yucheng Han, Xin Chen, Zebiao Huang, Bin Fu, and Gang Yu. AppAgent: Multimodal agents as smartphone users. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI)*, pages 1–20, 2025. doi: 10.1145/3706598.3713600.

Zhuosheng Zhang and Aston Zhang. You only look at screens: Multimodal chain-of-action agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3132–3149, 2024. doi: 10.18653/v1/2024.findings-acl.186.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded. In *International Conference on Machine Learning (ICML)*, 2024.

Shijie Zhou, Viet Dac Lai, Hao Tan, Jihyung Kil, Wanrong Zhu, Changyou Chen, and Ruiyi Zhang. GUI-AIMA: Aligning intrinsic multimodal attention with a context anchor for GUI grounding. arXiv preprint arXiv:2511.00810, 2025.

A Training and evaluation details

Data. AITW screenshots in the mirror we use are stored as raw RGB byte buffers without an image header; we decode them by matching the buffer length against seven observed resolutions (for example 540×1140 and 412×732) with a portrait-aspect fallback. Mind2Web screenshots are downscaled so that no image exceeds one megapixel. Examples are drawn from the AITW training stream in order, filtered to the mix's action labels; the first n eligible steps form the training slice and the next 250 (200 for the control mix) form validation.

Prompts. Stage 2 prompts contain the screenshot, the line `Goal: {goal}`, and the instruction `Predict the action coordinate.` (C uses `Predict the action type and coordinate.` and is trained to answer with the action word before the coordinate). D-token places `<|action_slot|>` at the start of the text; exactly one slot per sequence is asserted at every step.

Compute. Everything runs on single NVIDIA L4 GPUs. A Stage-2 run at 1,200 training examples takes about 35 minutes; the full study, including the runs that were later retracted, cost under 100 GPU-hours.

Optimization. AdamW with learning rate 2×10^{-5} and weight decay 0.01, batch size 1, gradient clipping at 1.0, two epochs, a fixed per-epoch shuffle seeded by the run seed. The action-embedding table (D-hook, D-token) and the auxiliary head (B) share the base learning rate. D-hook's table is zero-initialized; D-token's uses $\mathcal{N}(0, 0.02^2)$. Evaluation is greedy with at most 16 new tokens; the coordinate is parsed with a regular expression and an unparseable output counts as a miss at distance $\sqrt{2}$.

Statistics. For each comparison we pool per-example distances over the seeds that both variants share, resample the pooled units with replacement 10,000 times, and report the percentile interval and the two-sided bootstrap p-value of the mean difference; a paired sign-permutation test with 10,000 permutations is run alongside and agrees with the bootstrap in every case in the paper. Pooled units from the same example under different seeds are correlated, which makes the intervals somewhat optimistic; the permutation test is affected in the same way.

B Stage 1 and per-class tables

Table 5: Stage 1 action-type macro-F1 (three seeds). On Mind2Web the instruction names the action, so the screenshot adds little; on AITW the instruction describes a goal and the screenshot is what separates the classes.

Features	Mind2Web (2 classes)	AITW (5 classes)
text only	0.597 ± 0.011	0.141 ± 0.032
text + screenshot	0.605 ± 0.016	0.555 ± 0.036
zeroed (sanity)	0.469 ± 0.000	0.110 ± 0.051
TF-IDF + logistic regression	0.622	0.168
majority class	0.472	0.140

Table 6: Per-class hit@0.10 on the headline mix (five seeds). AITW *type* events carry a degenerate touch coordinate (mean distance $\sqrt{2}$ for every model), so the class is ungroundable and contributes a constant. The conditioning gain is a *click* gain; only D-hook also improves *scroll*.

Variant	click (n=169)	scroll (n=46)	type (n=35)
A: flat	0.256 ± 0.062	0.304 ± 0.034	0.000
B: aux. loss	0.357 ± 0.026	0.278 ± 0.018	0.000
C: hard routing	0.356 ± 0.065	0.172 ± 0.076	0.000
D-hook: additive	0.321 ± 0.064	0.357 ± 0.039	0.000
D-token: prepended	0.269 ± 0.069	0.304 ± 0.090	0.000

C Full scaling table

Table 7: Conditioning advantage vs. training-set size (AITW `all_with_coords`, 3 seeds per cell). Intervals are 95% cluster bootstraps over validation examples (all seeds of a sampled example kept together); stars from the same bootstrap. Absolute values are not comparable across rows because the validation slice moves with n; its click/scroll/type counts are given per row. The study is descriptive: 24 contrasts, no multiplicity correction.

n_{train}	val mix (c/s/t)	A hit@0.10	B − A	D-hook − A	B − A (hit@0.25)	D-hook − A (hit@0.25)
300	170/33/47	0.139 ± 0.032	+0.067 [+0.031, +0.103]***	+0.069 [+0.041, +0.099]***	-0.055 [-0.092, -0.019]**	+0.067 [+0.033, +0.101]***
500	198/12/40	0.315 ± 0.059	+0.011 [-0.019, +0.040]	+0.001 [-0.032, +0.033]	+0.004 [-0.027, +0.033]	+0.059 [+0.025, +0.092]***
800	141/71/38	0.261 ± 0.020	-0.040 [-0.073, -0.008]*	+0.048 [+0.009, +0.087]*	-0.039 [-0.069, -0.008]*	+0.064 [+0.025, +0.103]***
1200	169/46/35	0.255 ± 0.026	+0.051 [+0.021, +0.081]***	+0.045 [+0.015, +0.079]**	+0.071 [+0.036, +0.107]***	+0.055 [+0.017, +0.092]**
2500	166/53/31	0.244 ± 0.037	-0.012 [-0.039, +0.015]	+0.037 [+0.009, +0.067]**	-0.008 [-0.043, +0.028]	+0.104 [+0.067, +0.143]***
5000	187/29/34	0.363 ± 0.009	+0.005 [-0.025, +0.037]	+0.043 [+0.011, +0.076]**	-0.023 [-0.051, +0.005]	+0.032 [+0.001, +0.064]*

D Attention at every probe position

Table 4 reports the token that predicts the y coordinate. Table 8 gives the same measure for the last prompt token (the one that predicts the opening parenthesis) and for the token that predicts the first x digit, at the 3/4-depth layer. The last prompt token barely localizes under any conditioning, the x-predicting token localizes weakly, and the y-predicting token is where the target region receives attention.

Table 8: Share of image attention within 0.10 of the target (chance 0.026) at three probe positions, seed 42, 120 screens, 3/4-depth layer. Contrasts are gold minus condition with 95% paired-bootstrap intervals.

Model	Position	gold	wrong (cyclic)	wrong (click↔scroll)	zero
D-hook	last prompt token	0.040	0.036	0.035	0.036
D-hook	pre-x token	0.039	0.036	0.037	0.039
D-hook	pre-y token	0.111	0.064	0.049	0.119
D-token	last prompt token	0.038	0.034	0.034	0.035
D-token	pre-x token	0.038	0.034	0.033	0.035
D-token	pre-y token	0.095	0.080	0.079	0.079

E Qualitative examples

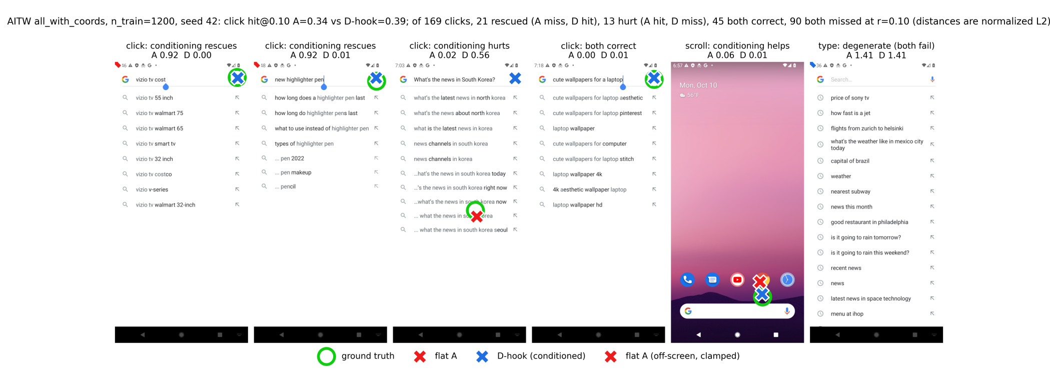

Figure 4: Predictions of the flat baseline (red) and D-hook (blue) against the target (green) on held-out AITW screens at the headline setting, one seed. The panels are a fixed spread of outcomes rather than a selection of wins: two rescues, a case where conditioning hurts, a case where both are right, a scroll, and a degenerate *type* example on which both models miss at distance $\sqrt{2}$. The title reports the base rates on this seed. In the rescue panels the baseline's prediction is off-screen and drawn clamped at the corner.

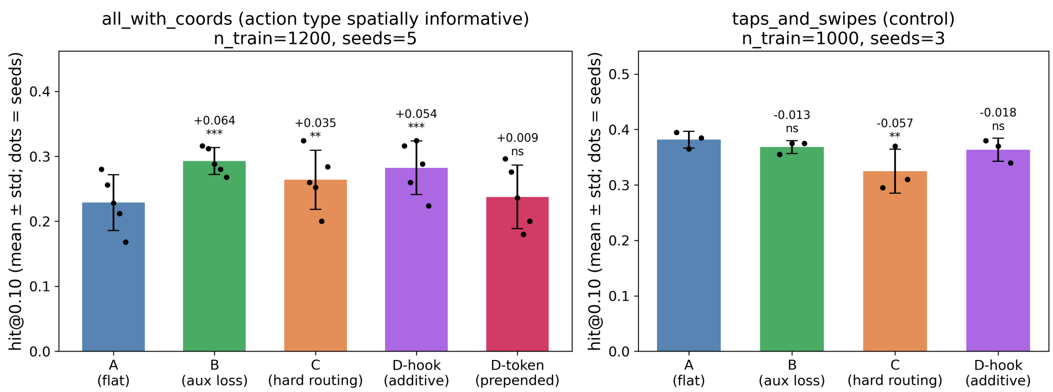

Figure 5: hit@0.10 per variant on the headline mix (left, five seeds) and the control mix (right, three seeds), with individual seeds as dots and the paired-bootstrap difference from the flat baseline above each bar.